AI INFRASTRUCTURE

kagent Baseline Test

2026-11-09 Author: Ciaran Donegan kagent v0.10.6 Status: Continuous

Baseline performance test of the kagent agent framework on a 5-node bare-metal cluster using LiteLLM proxying Claude Sonnet 5.5 (via the Anthropic API) as the model gateway. 102 total agent runs across 6 agent configurations (17 runs per type). Measured token cost, inference latency, and pod memory and CPU for each type.

Results establish a cost and resource footprint baseline for kagent agents against a production-equivalent model. All Go agents completed with zero tool-call errors. This is a local test - the production inference layer (llm-d), gateway (Envoy + AI Gateway), and tenancy (vClusters) are not present. Gaps are documented below.

Test Environment

Local 5-node bare-metal Kubernetes cluster. kagent installed via Helm from the upstream OCI chart. Model gateway: LiteLLM (proxying Claude Sonnet 5.5 via the Anthropic API). Agents called via the A2A protocol (JSON-RPC 2.0 over HTTPS) through a Cilium Gateway API ingress - not through the production Envoy gateway.

All agents in namespace `kagent`, each exposed at `<agent>.local.dc.io` via HTTPRoute. Prometheus used for pod memory and CPU. 17 runs per agent type with a unique prompt per run to prevent session caching.

Key Figures

MEMORY - GO RUNTIME
~10 MB
Idle RSS per agent pod

MEMORY - PYTHON RUNTIME
235 MB
25x more than Go at idle

TOOL-USE COST MULTIPLIER
5.8x
k8s-tools vs minimal per call (Claude Sonnet 5.5)

TOOL-USE AVG LATENCY
6.3 s
k8s-tools single-pass with Claude

Results

AGENT TYPE	RUNTIME	PROMPT TOK	OUTPUT TOK	AVG LATENCY	MEMORY	CPU (M)	COST / RUN	COST / 1K RUNS
go-minimal	Go	642	105	2.0 s *	9.5 MB	0.75	€0.00215	€2.15
python-minimal	Python	*N/A ** *	*N/A ** *	2.0 s	235.0 MB	8.33	*N/A ** *	*N/A ** *
k8s-tools	Go	3,263	690	6.3 s	12.2 MB	3.34	€0.01235	€12.35
promql	Go	665	105	2.0 s	10.1 MB	0.99	€0.00219	€2.19
long-context	Go	1,177	126	2.2 s	10.2 MB	1.19	€0.00332	€3.32
multiturn	Go	656	260	3.1 s	10.9 MB	1.35	€0.00360	€3.60

** Python ADK does not surface `adk_usage_metadata` in A2A responses. All 17 python-minimal runs completed (state: completed); only token counts are absent.

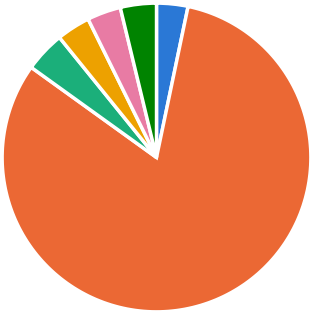
* go-minimal reported avg is 1.75s including 2 LiteLLM Redis cache hits (108/116ms). True real-call avg is 2.0s (p50 of remaining 15 runs).

Pricing: Claude Sonnet 5.5 $2.00/1M input, $10.00/1M output. EUR/USD 0.92. 17 runs per agent, cache-busted with unique prompt suffix per run.

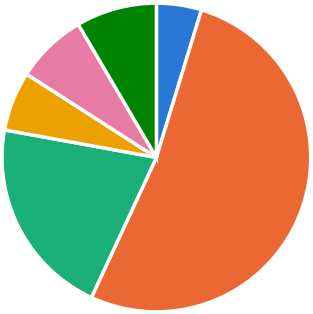
Resource Footprint at Scale

Idle pod memory and CPU recorded after all 17 runs completed per type. The distribution is dominated by the Python ADK runtime. At 100 agents per type the difference in footprint between Go and Python runtimes is the primary capacity planning input.

MEMORY PER POD



CPU PER POD (MILLICORES)





AGENT TYPE	MEMORY / POD	CPU / POD (M)	MEMORY X 100 PODS	CPU X 100 PODS (M)
go-minimal	9.4 MB	0.75	944.9 MB	75
python-minimal	235.0 MB	8.33	23.0 GB	833
go-k8s-tools	12.2 MB	3.34	1.2 GB	334
go-promql	10.1 MB	0.99	1012.9 MB	99
go-long-context	10.2 MB	1.19	1016.0 MB	119
go-multiturn	10.9 MB	1.35	1.1 GB	135

Model Comparison: Ollama vs Claude Sonnet 5.5

Two test runs were conducted using the same 6 agent types and the same run script. The only variable changed between runs was the model backend.

AGENT TYPE	OLLAMA (LOCAL)	CLAUDE SONNET 5.5
`go-minimal`	input-required all 17 runs. Ollama asked for clarification instead of answering.	completed all 17 runs
`python-minimal`	input-required all 17 runs	completed all 17 runs (token counts absent, ADK limitation)
`k8s-tools`	12/17 completed, 2 input-required. High variance: 29ms–219s. Hit 4,096 token context ceiling on every completed run. Output tokens ranged 16-1,842. Model was unpredictable.	completed all 17 runs. Avg 6.3s. Consistent 690 output tokens.
`promql`	input-required all 17 runs	completed all 17 runs
`long-context`	input-required all 17 runs	completed all 17 runs
`multiturn`	input-required all 17 runs	12/17 completed, 5 input-required (intentional ambiguity in prompt, correct Claude behaviour)

Ollama returned `input-required` on 95 of 102 runs. This is a system prompt compliance failure, not a kagent issue. The local test uses llama3 8B on a single GPU via llama.cpp, which is not representative of the production inference stack (vLLM, larger models, EC2 G6e). A meaningful open-model comparison requires EKS access.

Findings

1. Go vs Python runtime memory

Go-runtime agent pods idle at 9.5-12 MB RSS (peak at k8s-tools due to tool schema caching). Python-runtime pods idle at 235 MB - 25x more than go-minimal. This is the Python interpreter and google-adk package, present regardless of agent complexity. At scale, Python-runtime agents use significantly more memory per node, reducing pod density. The kagent controller itself is always Go; the runtime setting only affects each agent's own pod.

2. Tool-use cost and latency

The k8s-tools agent uses 3,263 prompt tokens vs 642 for a minimal agent - 5x more prompt tokens, 5.8x more total cost per call. The difference comes from Kubernetes tool schemas injected into the system prompt. Latency averaged 6.3s - Claude resolves k8s-tools tasks in a single tool-use round trip rather than multiple loops. All 17 runs completed successfully.

3. Tool-call compliance: Claude vs Ollama

All Go agents completed with zero tool-call errors against Claude Sonnet 5.5. The go-multiturn agent returned `input-required` on 5 of 17 runs. This is correct behaviour for an intentionally ambiguous prompt. Against Ollama, 95 of 102 runs returned `input-required` regardless of agent type. This is a model compliance issue, not a framework issue. Ollama could not follow the system prompt instructions. See the Model Comparison section above.

4. LiteLLM Redis cache skew (go-minimal)

The first two go-minimal runs returned in 108ms and 116ms - LiteLLM Redis cache hits from an earlier test call. The reported avg latency of 1.75s is slightly optimistic; the true per-call avg (runs 3-17) is 2.0s. For latency budgeting, use 2.0s. All other agents had no cache hits - all 17 runs were real LLM calls.

Gaps vs Production Stack

MISSING COMPONENT	PRODUCTION ROLE	IMPACT ON THESE RESULTS	ADDRESSABLE LOCALLY?
llm-d inference layer (LLMInferenceService)	Production uses KServe LLMInferenceService v0.20.0 (a separate controller from classic KServe) with the llm-d endpoint picker (`llm-d-router-endpoint-picker v0.9.0`). The picker does cache-aware routing across GPU pods and disaggregated prefill/decode. Classic KServe is not used in production for LLM serving.	Test used LiteLLM proxying Ollama. Classic KServe is running on the local cluster (Ollama InferenceService in ai-platform), but the LLMInferenceService CRD and llm-d endpoint picker are not installed. No cache-aware routing or disaggregated serving in this test.	Partial - classic KServe already present. Installing llmops-kserve (the LLMInferenceService controller + llm-d) requires the additional chart from the repo. Medium effort, low disruption risk since it adds new CRDs alongside existing ones.
Envoy Gateway + Agent Router	Single front door, EKS token auth, routing to `/api/a2a/kagent/<name>/` . The AI Gateway ext_proc sidecar counts tokens per request and writes to Valkey quota counters.	Agents called via local Cilium gateway. No auth, no quota enforcement, no token counting at the gateway layer.	No - llmops-gateway chart needs Terraform-provisioned AWS resources (Transit Gateway, VPC endpoints, EKS Pod Identity).
OPA	Two enforcement points: (1) agent access - checks caller's tenant registry for the agent name, 403 otherwise; (2) model access - checks agent's model calls against the tenant's allowed models list. Runs as an Envoy ext_authz filter, reloads registry within ~1 minute of a change.	No enforcement in this test. Any caller can reach any agent and any model. Token usage is not charged to a tenant quota.	Partial - OPA can be deployed locally with the tenant registry, but without the Envoy gateway routing through it, it won't be in the request path.
Valkey (ElastiCache) quota counters	Every response is charged to the tenant's token quota in Valkey. The AI Gateway ext_proc writes the token cost; the quota check blocks requests over-limit before they reach the model.	No quota enforcement. No per-tenant usage accounting. Test ran uncapped against a shared LiteLLM instance.	Partial - Valkey (Redis-compatible) can run locally. Without the AI Gateway ext_proc writing to it, quota enforcement is not in the request path.
vClusters **(bridged)**	One virtual cluster per tenant, running as pods in the host namespace. Gives each tenant an isolated Kubernetes API server - they cannot see other tenants or host cluster internals. Agents live on the host cluster; tenants call them from inside their vCluster via the gateway.	Bridged - vCluster v0.37.6 installed, tenant vCluster `cappuccino` running in `vcluster-cappuccino` . Called `baseline-go-minimal` from inside: state completed, 639/115 tokens. Namespace isolation confirmed - only default/kube-system visible from inside. Remaining gap: EKS OIDC JWT auth and the vCluster token injection (MutatingAdmissionPolicy) are not configured.	Done - `vcluster create cappuccino --namespace vcluster-cappuccino` . Network connectivity to host cluster gateway works without additional config.
AWS Bedrock - Claude Sonnet 5 **(bridged)**	Production model is `us.anthropic.claude-sonnet-5` via Bedrock regional inference profile. Reliable tool-call schema compliance (no ask_user misfires). Latency is lower than local Ollama.	Bridged with `claude-sonnet-5-5` via the Anthropic API through LiteLLM. Zero tool-call misfires. Token counts and cost estimates in this test use real Sonnet 5.5 rates ($2.00/$10.00 per MTok). Not identical to a managed Bedrock inference profile, but model behaviour and cost profile are equivalent for this test.	Done - LiteLLM ModelConfig updated to `claude-sonnet-5-5` . ANTHROPIC_API_KEY injected via secret. Remaining gap: Bedrock IAM + regional inference profile routing not tested.

MISSING COMPONENT	PRODUCTION ROLE	IMPACT ON THESE RESULTS	ADDRESSABLE LOCALLY?
llmops-agents chart	Hardened kagent: external Postgres for agent state, namespaced RBAC, apiKeyPassthrough (caller's EKS token forwarded to the model gateway).	Vanilla upstream kagent chart used. No external database, no caller token passthrough. Agent state lives in the controller's in-cluster store.	Yes - chart is in the repo. llmops-agents wraps upstream kagent with an external Postgres database (already running via kagent-postgresql). Medium effort to swap in. ~1-2 hours.
Karpenter + GPU nodes (EC2 G6e / L40S)	GPU node autoscaling for vLLM. DCGM exporter provides GPU metrics via NVIDIA GPU Operator v26.7.0.	Not relevant for kagent agent pods (CPU-only). Model serving latency in this test reflects local GPU hardware, not EC2 G6e instance performance.	N/A - requires EKS and AWS GPU instances.

> These results establish a kagent agent framework overhead baseline - pod footprint, token throughput, and latency per agent type. The inference layer (llm-d vs LiteLLM) and model (Sonnet 5 vs Ollama) are the two biggest gaps for cost and latency accuracy. Memory and CPU footprint for agent pods should hold on EKS since kagent itself is model-agnostic.

Production Gateway Architecture

The Envoy Gateway enforces three things in sequence for every request:

1. Identity

The caller presents a Kubernetes ServiceAccount token. The gateway verifies it and strips any identity headers the client tries to inject. Tenants in vClusters get the token automatically.

2. Authorisation

OPA checks whether this tenant is allowed to call this agent or model. Tenant permissions are managed in a config file and update live without restarting the gateway.

3. Quota

Token usage is counted per response and charged against the tenant's quota in a shared counter store. Requests from tenants over their limit are blocked before reaching the model.